Model Creation

Library Imports

```
ı and analysis through dataframes.
ıe data and processing it for the model.


ırray, and matrices.
ı structures efficiently,



· better analysis and understanding.


.ing.


.n-test split.
.dation and hyperparameter tuning as well.


:raining the data through the SVM model.
ıted in proper sequence.




ıance









)resenting the absence or presence of the feature (Scikit-learn, 2025).
```

```
# This part helps import the Spambase dataset by Hopkins et al. (1999).
# As for this dataset, it has no headers on the .data file.
df = pd.read_csv("spambase.data", header=None)

# As usual, the head() method is just to check and display.
# You can see here that there are no headers in the newly imported dataset.
df.head()
```

	0	1	2	3	4	5	6	7	8	9	...	48	49	50	
0	0.00	0.64	0.64	0.0	0.32	0.00	0.00	0.00	0.00	0.00	...	0.00	0.000	0.0	0.7
1	0.21	0.28	0.50	0.0	0.14	0.28	0.21	0.07	0.00	0.94	...	0.00	0.132	0.0	0.3
2	0.06	0.00	0.71	0.0	1.23	0.19	0.19	0.12	0.64	0.25	...	0.01	0.143	0.0	0.2
3	0.00	0.00	0.00	0.0	0.63	0.00	0.31	0.63	0.31	0.63	...	0.00	0.137	0.0	0.1
4	0.00	0.00	0.00	0.0	0.63	0.00	0.31	0.63	0.31	0.63	...	0.00	0.135	0.0	0.1

5 rows × 58 columns

```
# Since no headers are included in the database file, no column names are also
# As per the .names file, these are the column names for the data set with 48 a
column_names = [

    # 48 word frequency features as per the .documentation and the .names
    'word_freq_make', 'word_freq_address', 'word_freq_all', 'word_freq_3d',
    'word_freq_our', 'word_freq_over', 'word_freq_remove', 'word_freq_internet'
    'word_freq_order', 'word_freq_mail', 'word_freq_receive', 'word_freq_will',
    'word_freq_people', 'word_freq_report', 'word_freq_addresses',
    'word_freq_free', 'word_freq_business', 'word_freq_email', 'word_freq_you',
    'word_freq_credit', 'word_freq_your', 'word_freq_font', 'word_freq_000',
    'word_freq_money', 'word_freq_hp', 'word_freq_hpl', 'word_freq_george',
    'word_freq_650', 'word_freq_lab', 'word_freq_labs', 'word_freq_telnet',
    'word_freq_857', 'word_freq_data', 'word_freq_415', 'word_freq_85',
    'word_freq_technology', 'word_freq_1999', 'word_freq_parts',
    'word_freq_pm', 'word_freq_direct', 'word_freq_cs', 'word_freq_meeting',
    'word_freq_original', 'word_freq_project', 'word_freq_re', 'word_freq_edu',
    'word_freq_table', 'word_freq_conference',

    # 6 character frequency features as per the .documentation and the .names
    'char_freq_semicolon', 'char_freq_parenthesis', 'char_freq_bracket',
    'char_freq_exclamation', 'char_freq_dollar', 'char_freq_hash',

    # 3 capital run length features as per the .documentation and the .names
    'capital_run_length_average',
    'capital_run_length_longest',
    'capital_run_length_total',
```

```
    # Class label as per the .documentation and the .names
    'spam'
]

# By this part, we are including all the column names that we have manually set
# The same method is done again to simply present that the dataframe now has it
df.columns = column_names
df.head()
```

	word_freq_make	word_freq_address	word_freq_all	word_freq_3d	word_freq_our
0	0.00	0.64	0.64	0.0	0.32
1	0.21	0.28	0.50	0.0	0.14
2	0.06	0.00	0.71	0.0	1.23
3	0.00	0.00	0.00	0.0	0.63
4	0.00	0.00	0.00	0.0	0.63

5 rows × 58 columns

```
# This is just a simple check to know the number of rows (4,601) and columns (5
df.shape
```

```
(4601, 58)
```

```
# With this information, we can check the number of nulls and the total dataset
# While these have been included in the .documentation and the .names files, it
df.info()
```

```
 2   word_freq_all              4601 non-null   float64
 3   word_freq_3d               4601 non-null   float64
 4   word_freq_our              4601 non-null   float64
 5   word_freq_over             4601 non-null   float64
 6   word_freq_remove           4601 non-null   float64
 7   word_freq_internet         4601 non-null   float64
 8   word_freq_order            4601 non-null   float64
 9   word_freq_mail             4601 non-null   float64
 10  word_freq_receive          4601 non-null   float64
 11  word_freq_will             4601 non-null   float64
 12  word_freq_people           4601 non-null   float64
 13  word_freq_report           4601 non-null   float64
```

```
 21  word_freq_font              4601 non-null   float64
 22  word_freq_000               4601 non-null   float64
 23  word_freq_money             4601 non-null   float64
 24  word_freq_hp                4601 non-null   float64
 25  word_freq_hpl               4601 non-null   float64
 26  word_freq_george            4601 non-null   float64
 27  word_freq_650               4601 non-null   float64
 28  word_freq_lab               4601 non-null   float64
 29  word_freq_labs              4601 non-null   float64
 30  word_freq_telnet            4601 non-null   float64
 31  word_freq_857               4601 non-null   float64
 32  word_freq_data              4601 non-null   float64
 33  word_freq_415               4601 non-null   float64
 34  word_freq_85                4601 non-null   float64
 35  word_freq_technology        4601 non-null   float64
 36  word_freq_1999              4601 non-null   float64
 37  word_freq_parts             4601 non-null   float64
 38  word_freq_pm                4601 non-null   float64
 39  word_freq_direct            4601 non-null   float64
 40  word_freq_cs                4601 non-null   float64
 41  word_freq_meeting           4601 non-null   float64
 42  word_freq_original          4601 non-null   float64
 43  word_freq_project           4601 non-null   float64
 44  word_freq_re                4601 non-null   float64
 45  word_freq_edu               4601 non-null   float64
 46  word_freq_table             4601 non-null   float64
 47  word_freq_conference        4601 non-null   float64
 48  char_freq_semicolon         4601 non-null   float64
 49  char_freq_parenthesis       4601 non-null   float64
 50  char_freq_bracket           4601 non-null   float64
 51  char_freq_exclamation       4601 non-null   float64
 52  char_freq_dollar            4601 non-null   float64
 53  char_freq_hash              4601 non-null   float64
 54  capital_run_length_average  4601 non-null   float64
 55  capital_run_length_longest  4601 non-null   int64
 56  capital_run_length_total    4601 non-null   int64
 57  spam                        4601 non-null   int64
dtypes: float64(55), int64(3)
memory usage: 2.0 MB
```

```
# This is the checker for any duplicate rows, in summary.
# We can observe that there ar 391 rows with duplicates.
df.duplicated().sum()
```

```
np.int64(391)
```

```
# This step can be an unnecessary one, we simply add this to look at the duplic
# The only purpose for this step is to check the frequency of the duplicates.
df[df.duplicated()]
```

```
# This method drops the duplicates and re-assigns it to the dataframe.
# Dropping the rows ensures that we exclude bias on the duplicates during the m
df = df.drop_duplicates()
```

Data Loading

```
# As we load the data, we separate the features from the target variable.
# Axis=1 as the parameter is simply noting that we are dropping a column, not a
# In this dataset as we are looking at a spambase, our target variable is spam
X = df.drop('spam', axis=1)
y = df['spam']

# This is the usual train-test split with a ratio of 80:20.
# In order to truly compare the 3 models together, one split will be used all t
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
```

Data Modeling

```
ıive-Bayes.
:rain our first Naive-Bayes model.
```

```
l frequencies to the absent or present signals that the model assumes.
ent), while zeros are considered 0 (absent).
```

Data Training and Testing

```
# In here, we simply utilize a loop in order to present the classification repo
# A dataframe can be used here, but it is simply a decision to use print instea
# We iterate through the three pipelines that we created.
for name, pipe in pipelines.items():
    # This is simply a border that separates the new loop from the previous one
    # This part also shows the model name for less confusion.
    print("=" * 60)
    print(f"Model: {name}")

    # This is where we train each model using the training data.
    # The line after is for the prediction which tests the model's predictions
    pipe.fit(X_train, y_train)
    y_pred = pipe.predict(X_test)

    # This is where we calculate for the accuracy (how much the model is correc
    # We separate these two as main focusing metric for comparing the models.
    acc = accuracy_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)

    # This prints out the scores calculated for each metric.
    # Of course, the .4f is simply for showing four decimal points.
    print(f"Accuracy: {acc:.4f}")
    print(f"F1-score:  {f1:.4f}\n")

    # This part is the classification report that entails more details comparec
    # We simply print it to show an idea on the train and test phases through t
    print("Classification Report:")
    print(classification_report(y_test, y_pred))
```

```
============================================================
Model: GaussianNB
Accuracy: 0.8361
```

```
F1-score:  0.8226

Classification Report:
              precision    recall  f1-score   support

           0       0.96      0.76      0.85       506
           1       0.72      0.95      0.82       336

    accuracy                           0.84       842
   macro avg       0.84      0.86      0.84       842
weighted avg       0.87      0.84      0.84       842

============================================================
Model: MultinomialNB
Accuracy: 0.7886
F1-score:  0.7210

Classification Report:
              precision    recall  f1-score   support

           0       0.80      0.86      0.83       506
           1       0.76      0.68      0.72       336

    accuracy                           0.79       842
   macro avg       0.78      0.77      0.78       842
weighted avg       0.79      0.79      0.79       842

============================================================
Model: BernoulliNB
Accuracy: 0.8943
F1-score:  0.8658

Classification Report:
              precision    recall  f1-score   support

           0       0.90      0.92      0.91       506
           1       0.88      0.85      0.87       336

    accuracy                           0.89       842
   macro avg       0.89      0.89      0.89       842
weighted avg       0.89      0.89      0.89       842
```

Visualization

```
# This part is simply adding subplots to present all thre confusion matrices tc
# With side-by-side plots, we can easily compare and see the numbers on the cor
fig, ax = plt.subplots(1, 3, figsize=(13, 6))

# This was simply set as a counter.
# This is for displays using the index while the loop is traversing the array.
i=0
```

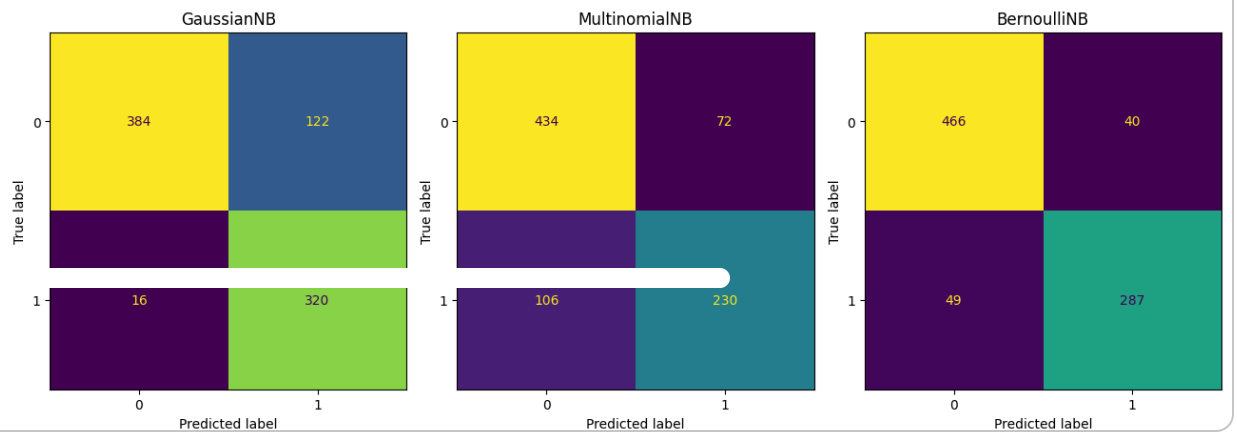
```
# This is the main loop that iterates again through the 3 models.
# As mentioned, they are stored in the pipelines array.
for name, pipe in pipelines.items():
  # This generates the confusion matrix that compates the actual target from th
  # The spam emails missed the the correct emails that were labeled as spam car
  cm = confusion_matrix(y_test, pipe.predict(X_test))

  # This aids in displaying the metrics of the confusion matrix but in a preser
  # As seen in the output below, the heatmap is easier to read and distinguish,
  disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=pipe.classe

  # This plots the heatmap generated in the code above, while disabling the col
  # The title is then set to avoid confusion between the three.
  disp.plot(ax=ax[i], values_format="d", colorbar=False)
  ax[i].set_title(f"{name}")

  # In addition to the earlier i=0 code, this increments the i variable.
  # Incrementing the i aids in displaying the next index inside the axes subplc
  i += 1

fig.tight_layout()
plt.show()
```




```
# This hold the figure plot to display the ROC curve for the 3 models.
# This has a bigger plot with no subplots as all three can be all seen as
plt.figure(figsize=(6, 5))

# The loop is again deployed to fetch and display the AUC.
# This loops inside the pipelines array once more.
for name, pipe in pipelines.items():

    # This is done again in order to make sure that AUC is calculated wel
    # Training is done agin in this line.
    pipe.fit(X_train, y_train)

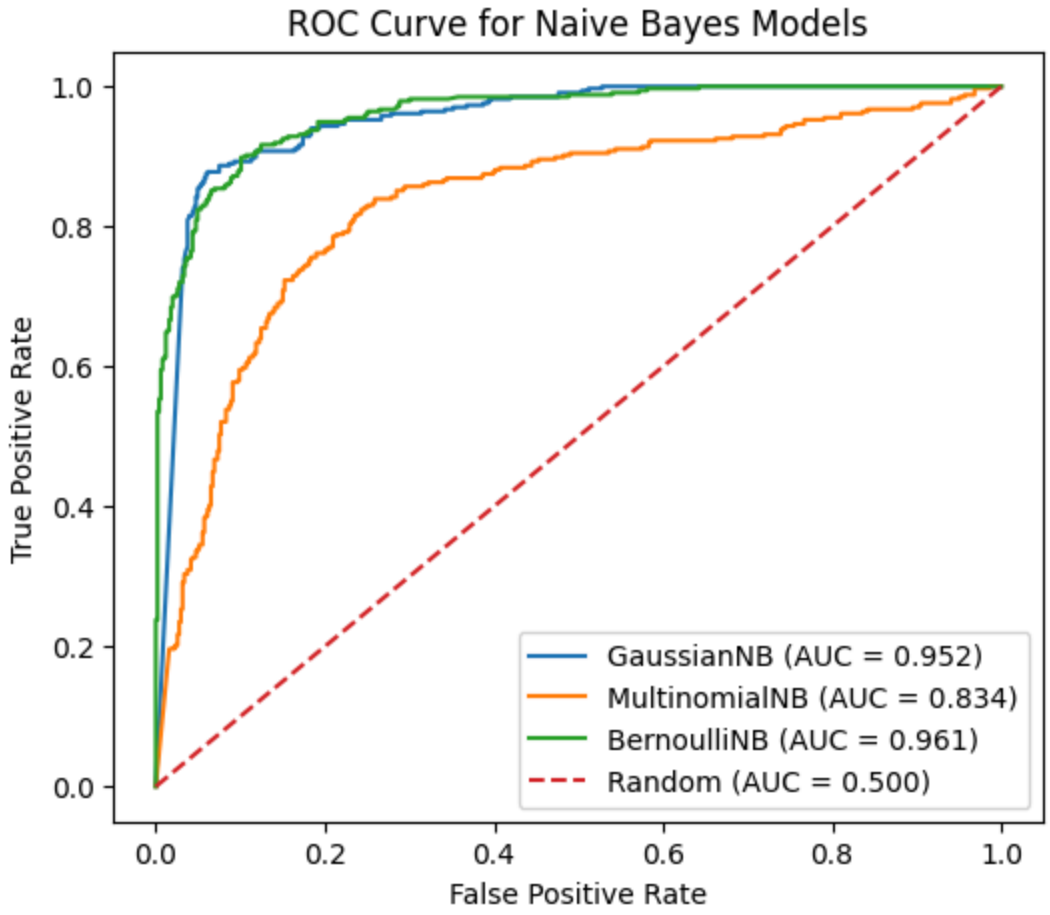
    # This gets the predicted targets of each model
    # The predict_proba provides raw probability for plotting the perform
    y_prob = pipe.predict_proba(X_test)[:, 1]
```

```
        # These two function calculate the true positive and false positive r
        # The results display the ability of the model to distinguish betweee
        fpr, tpr, _ = roc_curve(y_test, y_prob)
        roc_auc = auc(fpr, tpr)

        # This is for adding the label of the graph
        plt.plot(fpr, tpr, label=f"{name} (AUC = {roc_auc:.3f})")

# This part draws the random classifier inference.
# Basically, this is the baseline of how better the models are compared t
plt.plot([0, 1], [0, 1], linestyle="--", label="Random (AUC = 0.500)")

# These lines are simply for plotting the actual plot.
# Of course, it comes with its labels, titles, and legends.
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve for Naive Bayes Models")
plt.legend()
plt.show()
```



Discussion

Naive Bayes is another supervised machine learning model that provides prediction through probability distribution (GeeksforGeeks, 2017). In this activity, three (3) types of Naive Bayes Model have been created and evaluated. The dataset used in this activity is the Spambase dataset Hopkins et al. This provides three files including the actual dataset (.data), documentation (.documentation), and column names (.names).

Data Preparation and EDA

As part of the data preparation and the data exploration, the *.data* file was loaded with no headers into a dataframe. The column names were manually included and reviewed from the *.names* file, which were then loaded again to the dataframe. After double-checking using *shape()*, this resulted to 4601 rows and 58 columns of data.

Continuing with the exploration of the dataset and its cleaning, the shape() method was used to help familiarize with its contents. This revealed that 53 of the features are float and 3 are integers, one of which is the target variable (spam). The documentation and the .names file indicated these features are continuous that indicates that they are numerical in nature (Hoskin, 2018). As for the target, the values under these column are either 0 or 1, which despite being numerical are categorial on its own which also follows the documentation.

The same method confirms that there are no nulls in the dataset which eliminates the need to drop rows or set values to a default value. After this, the duplicated().sum() method was utilized to check for duplicates. This resulted to 391 rows of data, meaning that there is a need to drop the rows to avoid bias over the duplicated data. Before any row drops, the data duplicates are checked to inspect the frequency of duplicates. Nonetheless, these rows have been dropped to prepare for the data modeling later.

As for label encoding and the like, the entire dataset is numerical, removing the need for transforming them for Naive Bayes.

Data Loading

As usual, the dataset was loaded in preparation for training. Features are separated with the target variable which is "spam". The train-test split for this activity is 80:20.

Data Modeling

As mentioned, three types of Naive Bayes are modeled in this activity, with the goal of comparing them through the spambase dataset.

With the nature of Naive Bayes relying on probability distribution, there is no need to standardize the dataset. This is due to the *"naivety"* of the model to think that each feature is independent through probability, unlike other models relying on distance (Scikit-learn, 2019).

In order to look at the models of the types of Naive Bayes, the three were modeled and compared. Delving deep into the model types used, Gaussian is associated with continuous features following a normal distribution. Multinomial is for freuency baseed text features, while Bernoulli emphasizes the presence of words, as it deals with binary features (GeeksforGeeks, 2017).

Results and Visualization

Among the three models, Bernoulli performed best in both accuracy of **89.43%** and F1-score of **86.58%**. The confusion matrix also shows that 466 non-spam emails and 287 spam emails are correctly classified, along with 40 false positives and 49 false negatives that presents the best balance. This tells us that the presence of certain words become an indicator for detecting spam. For the ROC curve visualization, the Bernoulli line is the highest and closest to the top left corner with a **0.961 AUC**. This supports the confusion matrix results that displays the high rate of catching true positives while keeping the false positives low.

Next second best performer is Gaussian with **83.61%** accuracy and **82.26%** F1-Score. Nonetheless, these results present the model's excellence in classifying spam emails. In terms of the confusion matrix, only 16 spam emails were not classified correctly, however there is a concerning high rate of false positives, which was 122. As per the ROC curve, it is almost as high as Bernoulli, falling off at **0.952 AUC**.

As for the Multinomial, it has the lowest accuracy at **78.86%** and **72.10%** F1-Score. The confusion matrix also shows it has the most false negatives that flagged 106 spam emails as non-spam. Its low performance also stands out with the ROC curve further with a low line that is closest to the random line with a **0.834 AUC**.

Overall, in this Spambase dataset, Bernoulli being the best performer gives an idea that **trigger words (as Bernoulli is based on presence/absence of a word) in emails is the most effective predictor of the email's class**. While Gaussian is also a good predictor but not the best, this suggests that it is less precise as the features of a spam

email do not always follow the normal distribution that the model type assumes. Additionally with the multinomial being the lowest, this indicates that frequency alone in this dataset is not intuitive as the Bernoulli's.

References

- GeeksforGeeks. (2017, March 3). Naive Bayes Classifiers. GeeksforGeeks. https://www.geeksforgeeks.org/machine-learning/naive-bayes-classifiers/
- Hopkins, M., Reeber, E., Forman, G., & Suermondt, J. (1999). Spambase [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C53G6X.
- Hoskin, T. (2018). Data Types. https://www.mayo.edu/research/documents/data-types/doc-20408956
- Scikit-learn. (2019). 1.9. Naive Bayes — scikit-learn 0.21.3 documentation. Scikit-Learn.org. https://scikit-learn.org/stable/modules/naive_bayes.html
- Scikit-learn. (2025). sklearn.preprocessing.Binarizer. Scikit-Learn. https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.Binarizer.html

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.